P-Grafo: arquitectura C4, diseño hexagonal y plan Scrum

Objetivo

P-Grafo se plantea como un sistema de predicción estructural de proteínas orientado a CPU, con núcleo desacoplado de infraestructura y preparado para evolucionar desde un MVP en Python hasta una plataforma extensible de investigación.[cite:6242][cite:6245][cite:6251]

La arquitectura propuesta combina el modelo C4 para documentación y comunicación de arquitectura con arquitectura hexagonal para implementación, de modo que el dominio y los casos de uso permanezcan independientes de frameworks, base de datos, CLI, API o motores de cómputo externos.[cite:6242][cite:6245][cite:6249][cite:6251]

Principios de diseño

- Núcleo de dominio puro en Python, sin dependencias de framework ni de infraestructura.[cite:6250][cite:6251]
- Dependencias apuntando hacia dentro, siguiendo inversión de dependencias y separación de responsabilidades.[cite:6242][cite:6249][cite:6250]
- Puertos explícitos para entrada y salida; adaptadores reemplazables para CLI, API, almacenamiento, MSA, DCA y exportación de resultados.[cite:6242][cite:6249][cite:6251]
- Documentación arquitectónica jerárquica mediante C4: contexto, contenedores, componentes y código.[cite:6245]
- MVP pequeño primero, backlog incremental después, alineado con ciclos Scrum de planificación, sprint, review y retrospectiva.[cite:6243][cite:6248]

C4 nivel 1: contexto

Sistema

P-Grafo es un sistema de investigación bioinformática que recibe una secuencia de proteína, construye evidencia evolutiva y estructural, estima restricciones geométricas y produce una estructura 3D candidata junto con métricas de confianza y artefactos exportables.[cite:6245]

Actores externos

- Investigador bioinformático: lanza predicciones, compara resultados y analiza estructuras.
- Desarrollador del proyecto: incorpora nuevos adaptadores y algoritmos.
- Herramientas externas de secuencia/alineamiento: p. ej., HHblits o JackHMMER, integradas como adaptadores de infraestructura.

- Repositorios de datos estructurales/biológicos: PDB, UniRef u otras fuentes que alimenten entrenamiento o validación.

Relaciones

- El investigador usa CLI o API para crear una predicción.
- P-Grafo consulta motores externos para MSA/DCA cuando corresponda.
- P-Grafo persiste resultados y metadatos en almacenamiento local o futuro backend.
- P-Grafo exporta PDB/JSON/reportes para análisis posterior.

C4 nivel 2: contenedores

Contenedor 1: CLI / API de aplicación

Es la puerta de entrada inicial del MVP. Debe validar entrada, invocar casos de uso y presentar resultados sin contener lógica científica de negocio.[cite:6242][cite:6251]

Contenedor 2: núcleo de aplicación

Contiene casos de uso, orquestación del pipeline, DTOs, políticas de ejecución y puertos. Este contenedor coordina las fases de P-Grafo sin conocer implementaciones concretas de infraestructura.[cite:6242][cite:6250][cite:6251]

Contenedor 3: dominio

Aloja entidades, value objects, reglas científicas centrales y servicios de dominio relacionados con secuencias, residuos, restricciones geométricas, grafos de contactos y modelos de estructura.[cite:6250][cite:6251]

Contenedor 4: infraestructura científica

Implementa adaptadores para MSA, DCA, persistencia, serialización, logging, visualización y eventual integración con librerías numéricas o herramientas externas.[cite:6242][cite:6249][cite:6251]

Contenedor 5: almacenamiento y artefactos

En el MVP puede ser sistema de ficheros local para entradas, resultados, configuraciones y estructuras exportadas. Más adelante puede evolucionar a base de datos de experimentos y caché de ejecuciones.[cite:6244][cite:6251]

C4 nivel 3: componentes

Dentro del núcleo de aplicación

Componente	Responsabilidad
`RunPredictionUseCase`	Ejecutar pipeline completo de predicción.
`BuildMsaUseCase`	Orquestar construcción o recuperación de MSA.
`ComputeCouplingsUseCase`	Generar matriz de coevolución/restricciones.
`BuildGraphUseCase`	Construir grafo proteico desde secuencia y evidencia.
`PredictDistancesUseCase`	Estimar distancias/ángulos con modelo base.
`OptimizeStructureUseCase`	Convertir restricciones en coordenadas 3D.
`ExportStructureUseCase`	Generar PDB/JSON/resumen del experimento.
`ExperimentQueryService`	Consultar ejecuciones previas y metadatos.

Puertos de entrada

- `PredictionInputPort`
- `ExperimentQueryInputPort`
- `ModelTrainingInputPort` (fase posterior)

Puertos de salida

- `MsaProviderPort`
- `CouplingComputerPort`
- `DistancePredictorPort`
- `StructureOptimizerPort`
- `StructureRepositoryPort`
- `ExperimentRepositoryPort`
- `StructureExporterPort`
- `LoggerPort`
- `ConfigPort`

Dentro del dominio

Componente	Responsabilidad
`ProteinSequence`	Validación y representación de secuencia primaria.
`Residue`	Datos y propiedades por residuo.
`MultipleSequenceAlignment`	Representación del alineamiento múltiple.
`CouplingMatrix`	Evidencia de coevolución entre residuos.

Componente	Responsabilidad
`ProteinGraph`	Nodos, aristas, pesos y anotaciones del grafo.
`DistanceConstraintSet`	Restricciones geométricas derivadas.
`StructureModel`	Coordenadas, energía, score y metadatos.
`PredictionJob`	Agregado raíz de una ejecución.
`StructureQualityService`	Reglas de scoring, consistencia y validación.

Adaptadores iniciales para el MVP

- `CliPredictionController`
- `FilesystemStructureRepository`
- `FilesystemExperimentRepository`
- `SubprocessMsaAdapter`
- `PlmDcaAdapter` o adaptador provisional simulado
- `XgboostDistancePredictorAdapter` o baseline alternativo
- `ScipyStructureOptimizerAdapter`
- `PdbExporterAdapter`
- `JsonReportExporterAdapter`

C4 nivel 4: código y módulos sugeridos

Estructura de paquetes

```
p-grafo/
├── README.md
├── docs/
│   ├── architecture/
│   │   ├── c4-context.md
│   │   ├── c4-container.md
│   │   ├── c4-component.md
│   │   └── decisions/
│   ├── scrum/
│   │   ├── roadmap.md
│   │   ├── backlog.md
│   │   └── definition-of-done.md
│   └── research/
├── src/
│   └── p_grafo/
│       ├── domain/
│       │   ├── entities/
│       │   ├── value_objects/
│       │   ├── services/
│       │   └── exceptions/
│       ├── application/
```

```
│       │   ├── use_cases/
│       │   ├── ports/
│       │   │   ├── inbound/
│       │   │   └── outbound/
│       │   ├── dto/
│       │   └── services/
│       ├── infrastructure/
│       │   ├── msa/
│       │   ├── dca/
│       │   ├── predictors/
│       │   ├── optimizers/
│       │   ├── persistence/
│       │   ├── exporters/
│       │   ├── config/
│       │   └── logging/
│       ├── adapters/
│       │   ├── cli/
│       │   ├── api/
│       │   └── batch/
│       └── bootstrap/
│           └── container.py
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── contract/
│   └── e2e/
├── data/
│   ├── samples/
│   ├── fixtures/
│   └── outputs/
├── scripts/
├── pyproject.toml
├── .github/
│   └── workflows/
└── Makefile
```

La separación entre dominio, aplicación e infraestructura sigue la orientación hexagonal y facilita independencia de UI, base de datos y frameworks.[cite:6242][cite:6244][cite:6250][cite:6251]

Diseño hexagonal propuesto

Centro del hexágono

En el centro estarán el dominio y los casos de uso. Nada dentro del núcleo importará librerías de CLI, FastAPI, pandas, xgboost, scipy, subprocess ni almacenamiento directo.[cite:6242][cite:6249][cite:6250][cite:6251]

Puertos de entrada

Los puertos de entrada expresan lo que el sistema sabe hacer:

- Ejecutar predicción completa.
- Consultar una ejecución.
- Exportar resultados.
- Entrenar o recalibrar modelo base de distancias.

Puertos de salida

Los puertos de salida expresan lo que el núcleo necesita del exterior:

- Obtener/alojar MSA.
- Calcular DCA u otra matriz de acoplamientos.
- Predecir restricciones geométricas.
- Optimizar estructura 3D.
- Persistir estructuras y metadatos.
- Exportar a formatos científicos.
- Registrar eventos y telemetría.

Adaptadores de entrada

- CLI para el MVP.
- API HTTP en una fase posterior.
- Batch runner para experimentos reproducibles.
- Notebook adapter solo como capa externa, nunca como lugar de lógica de negocio.

Adaptadores de salida

- File system.
- Wrappers de herramientas externas.
- Baseline predictor.
- Motor de optimización.
- Reportes/exportadores.

Decisiones técnicas para el MVP en Python

Framework base

- Python 3.12.
- Gestión de dependencias con `uv` o `poetry`.
- `src/` layout para evitar imports accidentales y ordenar paquetes, una práctica común en proyectos Python bien estructurados.[cite:6244]

Librerías iniciales

- `pydantic` para DTOs de borde si se desea validación estricta en adaptadores.
- `networkx` o estructura propia ligera para el grafo inicial.
- `numpy` para tensores y matrices básicas.
- `scipy` para optimización.
- `xgboost` o `scikit-learn` para predictor base de distancias del MVP.
- `pytest` para pruebas.
- `typer` para CLI limpia.
- `rich` para salida amigable de experimentos.

Estrategia de testing

- Unit tests en dominio y casos de uso.
- Contract tests para cada puerto/adaptador.
- Integration tests para pipeline mínimo con datos de juguete.
- E2E tests para comando CLI completo sobre una secuencia corta.

Plan Scrum de implementación

Scrum recomienda trabajar por backlog, planificación, ejecución de sprint, revisión y retrospectiva; para un MVP científico conviene mantener iteraciones cortas con incrementos técnicamente ejecutables.[cite:6248]

Además, antes del backlog detallado conviene tener una arquitectura de información y flujo de tareas básico para que historias y épicas no nazcan desalineadas con el producto real.[cite:6243]

Roles sugeridos

- Product Owner: prioriza valor científico y entregables del MVP.
- Scrum Master: protege el proceso y elimina bloqueos.
- Development Team: bioinformática, arquitectura, Python, validación científica.

Cadencia sugerida

- Sprint de 2 semanas.
- Daily de 15 minutos.
- Sprint review con demo funcional.
- Retrospectiva al final de cada sprint.

Product backlog inicial

Épica 1: base arquitectónica

- Como desarrollador, quiero un repositorio inicial con estructura hexagonal para poder empezar sin deuda estructural.
- Como equipo, quiero diagramas C4 versionados para alinear decisiones técnicas.
- Como equipo, quiero CI básica para ejecutar lint, tests unitarios y checks de packaging.

Épica 2: dominio proteico

- Como sistema, quiero representar secuencias, residuos y estructuras como objetos de dominio válidos.
- Como sistema, quiero validar secuencias de entrada antes de arrancar cómputos costosos.

Épica 3: pipeline científico mínimo

- Como usuario, quiero ejecutar una predicción mínima desde CLI con una secuencia FASTA.
- Como sistema, quiero construir un grafo proteico básico desde features simples.
- Como sistema, quiero optimizar una estructura 3D con restricciones iniciales.

Épica 4: persistencia y exportación

- Como usuario, quiero guardar resultados en PDB y JSON.
- Como usuario, quiero consultar metadatos de ejecuciones previas.

Épica 5: calidad y reproducibilidad

- Como equipo, quiero fixtures reproducibles y seeds fijas.
- Como equipo, quiero pruebas de contrato sobre puertos y adaptadores.

Roadmap por sprints

Sprint 0: preparación

Objetivo: dejar listo el terreno de ingeniería.

- Crear repositorio GitHub.
- Configurar ramas, protección y convenciones.
- Crear esqueleto hexagonal.
- Añadir README inicial, licencia, CODEOWNERS y `.editorconfig`.
- Preparar CI con lint + tests vacíos.
- Redactar C4 contexto y contenedores.

Entregable: repo inicial ejecutable con arquitectura documentada.

Sprint 1: núcleo de dominio

Objetivo: crear el lenguaje del dominio.

- Implementar `ProteinSequence`, `Residue`, `ProteinGraph`, `StructureModel`.
- Crear puertos principales.
- Añadir primeros unit tests.
- Definir DTO de entrada/salida para predicción.

Entregable: dominio estable y probado.

Sprint 2: caso de uso MVP

Objetivo: correr pipeline mínimo de punta a punta con adaptadores simulados.

- `RunPredictionUseCase`.
- Adaptador CLI con FASTA simple.
- Adaptadores fake para MSA/DCA.
- Optimizador inicial sencillo.
- Exportación JSON/PDB básica.

Entregable: demo end-to-end con datos sintéticos.

Sprint 3: primer pipeline científico real

Objetivo: sustituir mocks por cómputo útil.

- Integrar MSA real o semirreal.
- Integrar cálculo inicial de acoplamientos.
- Construir grafo a partir de evidencia real.
- Añadir baseline de predicción de distancias.

Entregable: primera predicción técnicamente significativa.

Sprint 4: calidad y observabilidad

Objetivo: hacer el MVP mantenible y repetible.

- Logging estructurado.
- Métricas de pipeline.
- Contract tests adaptadores.
- Fixtures reproducibles.
- Documentar decisiones de arquitectura.

Entregable: MVP reproducible y trazable.

Sprint 5: endurecimiento del MVP

Objetivo: preparar primera versión utilizable por terceros.

- Mejora de UX de CLI.
- Manejo de errores.
- Reporte resumido de resultados.
- Empaquetado instalable.
- Guía de contribución.

Entregable: `v0.1.0` del MVP en Python.

Definition of Done sugerida

Una historia se considera terminada cuando:

- Tiene tests adecuados.
- Pasa CI.
- Respeta puertos y adaptadores sin acoplar el núcleo a infraestructura.
- Incluye documentación mínima.
- No rompe diagramas C4 ni contratos existentes.
- Tiene criterio científico verificable si afecta al pipeline.

Plan para el repositorio GitHub

Estructura inicial recomendada

- `main`: rama protegida.
- `develop` opcional si queréis flujo GitFlow ligero; si sois pocos, trunk-based con ramas cortas también sirve.
- Plantillas de PR e issues.

- Labels: `architecture`, `domain`, `application`, `infra`, `scientific`, `good first issue`, `tech debt`.

Ficheros iniciales

- `README.md`
- `LICENSE`
- `.gitignore`
- `.editorconfig`
- `pyproject.toml`
- `Makefile`
- `.pre-commit-config.yaml`
- `.github/workflows/ci.yml`
- `docs/architecture/*`
- `docs/scrum/*`

README mínimo

Debe incluir:

- Problema que resuelve P-Grafo.
- Principios arquitectónicos.
- Mapa del repo.
- Cómo arrancar localmente.
- Cómo ejecutar tests.
- Roadmap del MVP.

Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Mezclar lógica científica con CLI o notebooks	Alto	Forzar puertos y casos de uso desde el día 1.
Acoplarse demasiado pronto a una herramienta externa	Alto	Crear adaptadores intercambiables y dobles de prueba.
Sobre-diseñar antes del MVP	Medio	Limitar Sprint 0 y Sprint 1 a lo esencial.
Falta de validación científica temprana	Alto	Introducir fixtures de referencia y métricas desde Sprint 2.
Crecimiento caótico del código	Alto	C4 + ADRs + DoD + CI.

Orden recomendado de ejecución inmediata

1. Crear repo GitHub y estructura base.
2. Subir documentación inicial: visión, C4 contexto, C4 contenedores, backlog, roadmap.
3. Implementar esqueleto hexagonal en `src/p_grafo/`.
4. Añadir CLI mínima y caso de uso `RunPredictionUseCase` con adaptadores fake.
5. Validar que el pipeline entra por adaptador, atraviesa puertos y sale por repositorio/exportador.
6. Solo entonces empezar el MVP científico real en Python.

Siguiente paso práctico

El siguiente entregable lógico es crear el esqueleto del repositorio y los documentos base (`README`, `docs/architecture`, `pyproject.toml`, `src/p_grafo/...`) para que luego el MVP se construya sobre una base estable y no al revés.[cite:6244][cite:6247]